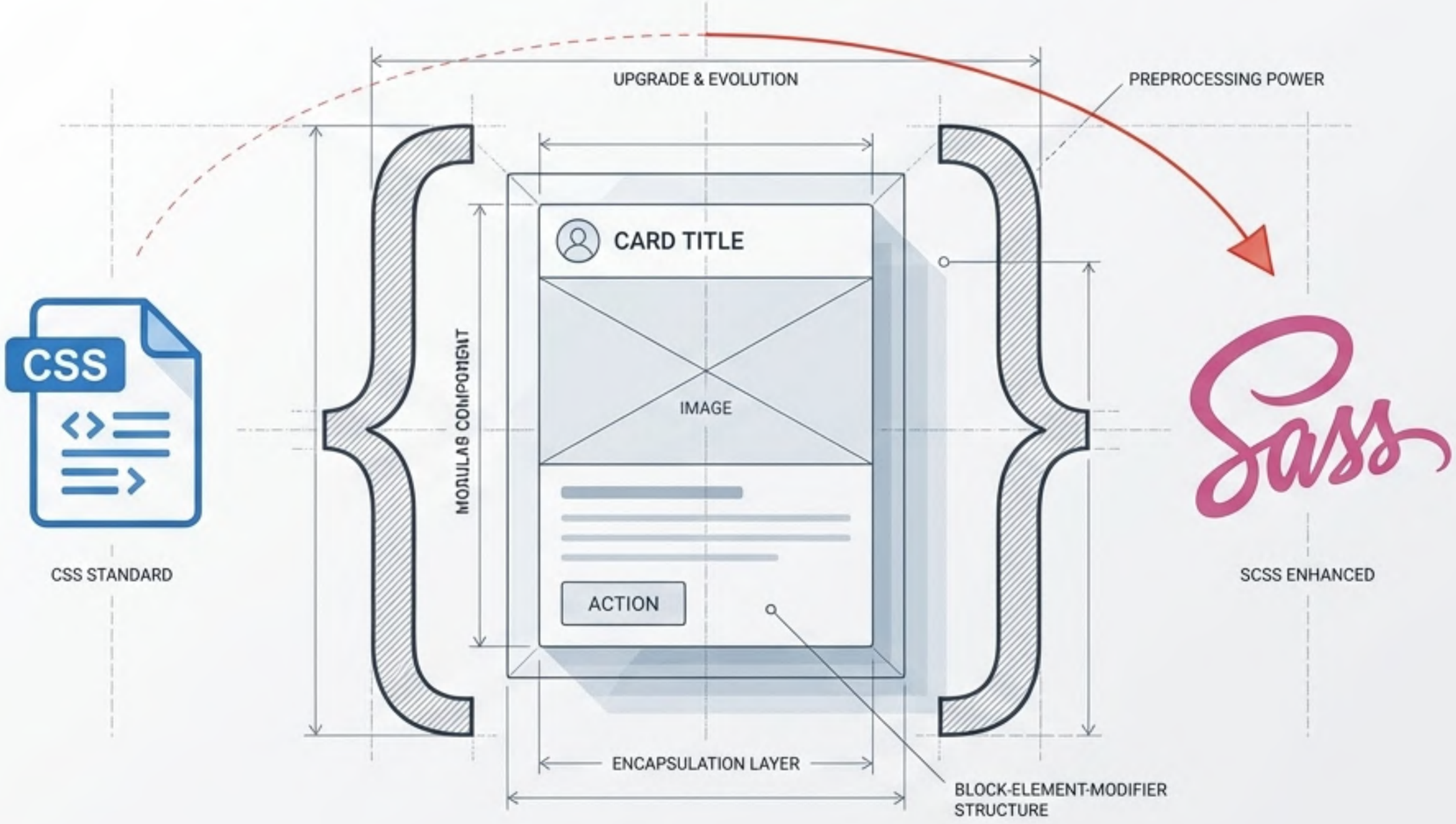
SCSS + BEM: CSS CON SUPERPODERES

Domina el preprocesador que revolucionará tu forma de escribir código modular



NotebookLM

El Dolor del BEM Tradicional

BEM es excelente para la organización, pero el CSS plano te obliga a repetir el nombre del bloque una y otra vez. El resultado: fatiga, errores tipográficos y archivos gigantescos.

```
1
2   .tarjeta__contenedor { padding: 20px; }
3
4   .tarjeta__imagen { width: 100%; }
5
6   .tarjeta__titulo { font-size: 24px; }
7
8   .tarjeta__boton { border-radius: 4px; }
9
10  .tarjeta__boton--activo { background: blue;
11
12  }
13
14
15
```

Nivel de Fatiga

Causa principal: Duplicación excesiva de selectores.

NotebookLM

CSS vs. SCSS: ¿Por qué dar el salto?

SCSS es un preprocesador. Escribes código avanzado y él lo traduce a CSS normal para el navegador.

Característica	CSS Plano	SCSS (Sass)
Repetición de Selectores BEM	Alta (Escribes todo)	Nula (Uso del operador &)
Variables	Nativas, pero complejas	Simples, matemáticas y color
Reutilización de Lógica	Ninguna	Alta (Uso de Mixins)
Curva de Aprendizaje	Base	Baja (CSS válido es SCSS)

El Estándar de la Industria

NotebookLM

Superpoder 1: Variables (El Poder de la Memoria)

Almacena colores, tipografías o tamaños en un solo lugar.
Si el diseño cambia, actualizas una línea de código, no cien.

```
1
2
3  $color-primario: #FF0055;
4  $fuente-base: 'Inter', sans-serif;
5
6  $radio-borde: 8px;
7
8
9
10
```

Tarjeta de Ejemplo

Este es el contenido de la tarjeta.
Se adapta con variables.

Acción Principal

SCSS compila esto directamente a valores estáticos. ¡100% de compatibilidad en navegadores!

NotebookLM

Superpoder 2: Anidamiento (Nesting)

El fin de repetir selectores. Escribe tus estilos con la misma jerarquía visual que tu HTML.

CSS Tradicional (Plano)

```
1  .nav { background: black; }
2  .nav ul { margin: 0; }
3  .nav li { display: inline; }
```

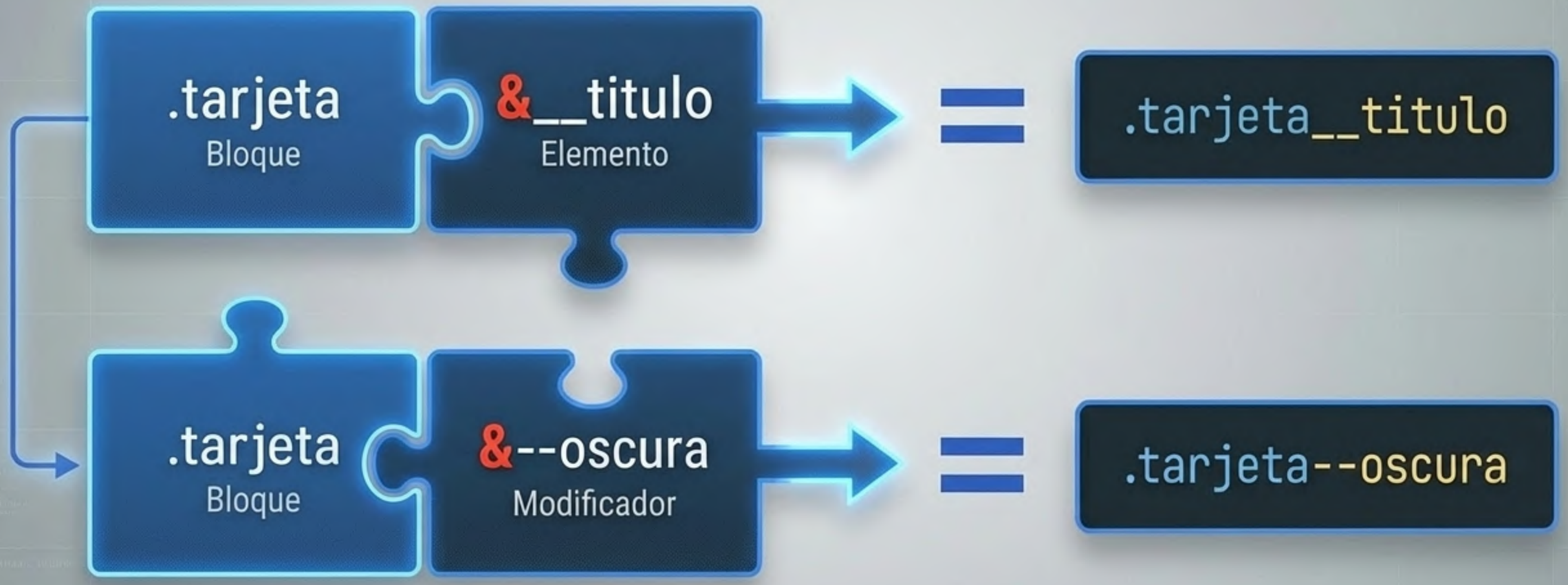
SCSS (Jerárquico)

```
1  .nav {
2      background: black;
3      ul { margin: 0; }
4      li { display: inline; }
5  }
```

NotebookLM

El Operador Mágico: &

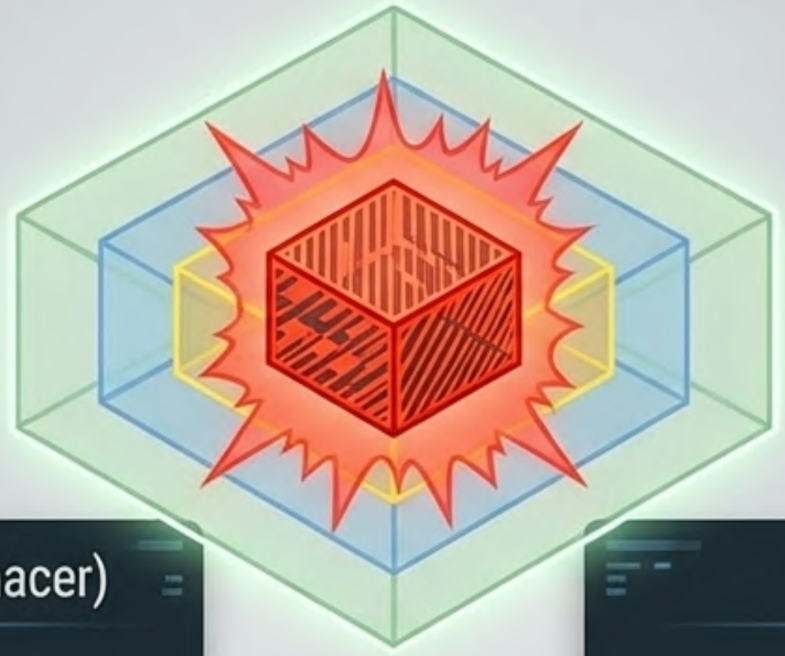
El símbolo & (Ampersand) es la clave para usar BEM en SCSS. Funciona como una pieza de rompecabezas que toma el nombre del bloque padre y lo fusiona dinámicamente.



NotebookLM

Mejores Prácticas: La Regla de la Inception

Anidar es adictivo, pero peligroso. Un anidamiento profundo genera selectores CSS gigantescos, lentos y difíciles de sobrescribir.



REGLA DE ORO:
¡Nunca anides más de 3 niveles de profundidad!

El Anti-Patrón (Lo que NO debes hacer)

```
.tarjeta {
  &__cuerpo {
    ul { li { a { color: red; } } } }
  }
}
```

CSS Compilado (Hinchado)

```
.tarjeta__cuerpo ul li a { color: red; }
```

NotebookLM

Superpoder 3: Mixins (Tu Arsenal de Código)

¿Cansado de escribir las mismas tres líneas para centrar elementos con Flexbox?
Crea un @mixin y úsalo como un sello donde lo necesites.

El Molde (Definición)

```
@mixin flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

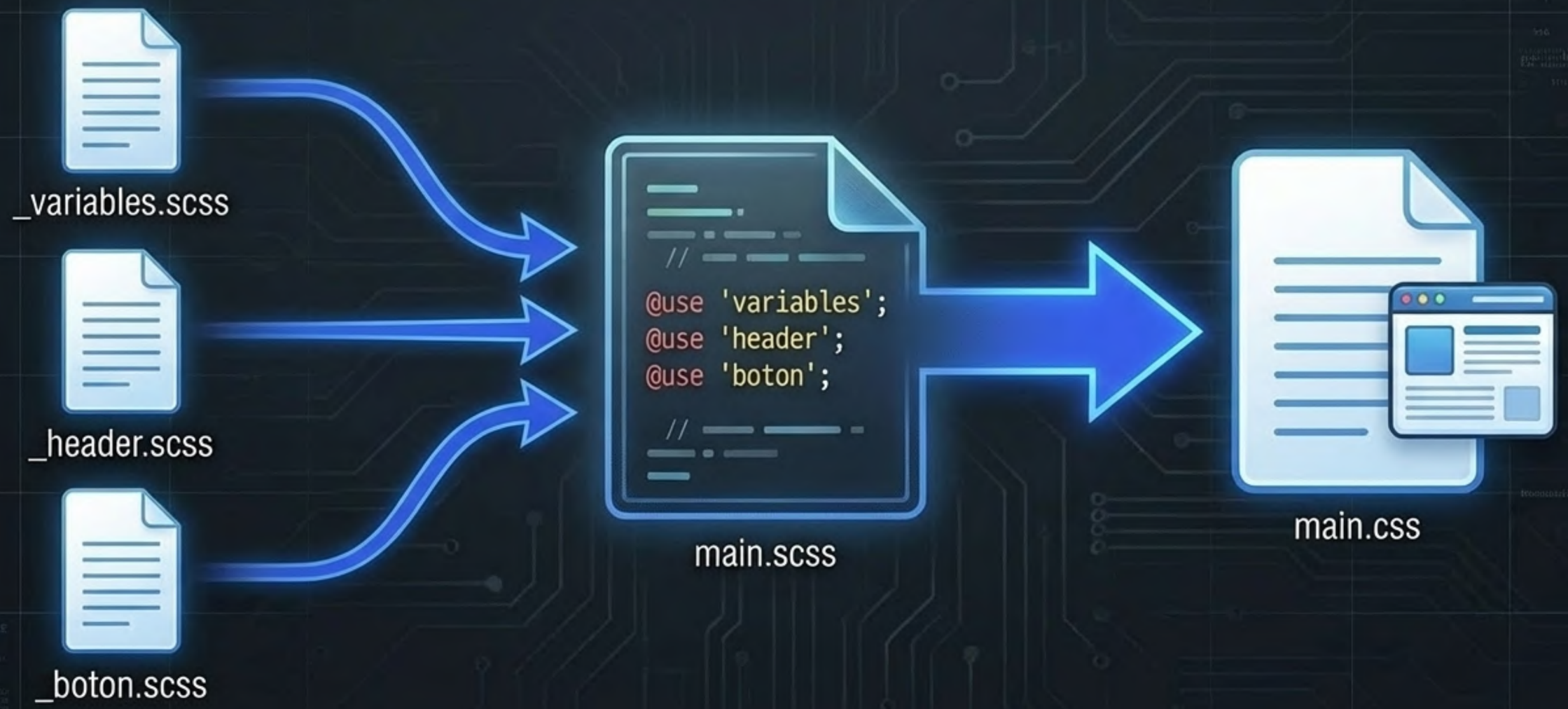
El Sello (Uso)

```
.cabecera {
  @include flex-center;
}
```

NotebookLM

Superpoder 4: Arquitectura Modular

Divide y vencerás. Usa archivos parciales (con un guión bajo _) para separar tus bloques BEM. SCSS los unirá todos en un solo archivo final.



NotebookLM

Proyecto Práctico: Menú de Hamburguesa Animado

Es hora de la batalla final. Combinaremos Variables, Anidamiento, el operador &
y Modificadores BEM para crear la animación UI más solicitada.

Superpoderes a Utilizar:

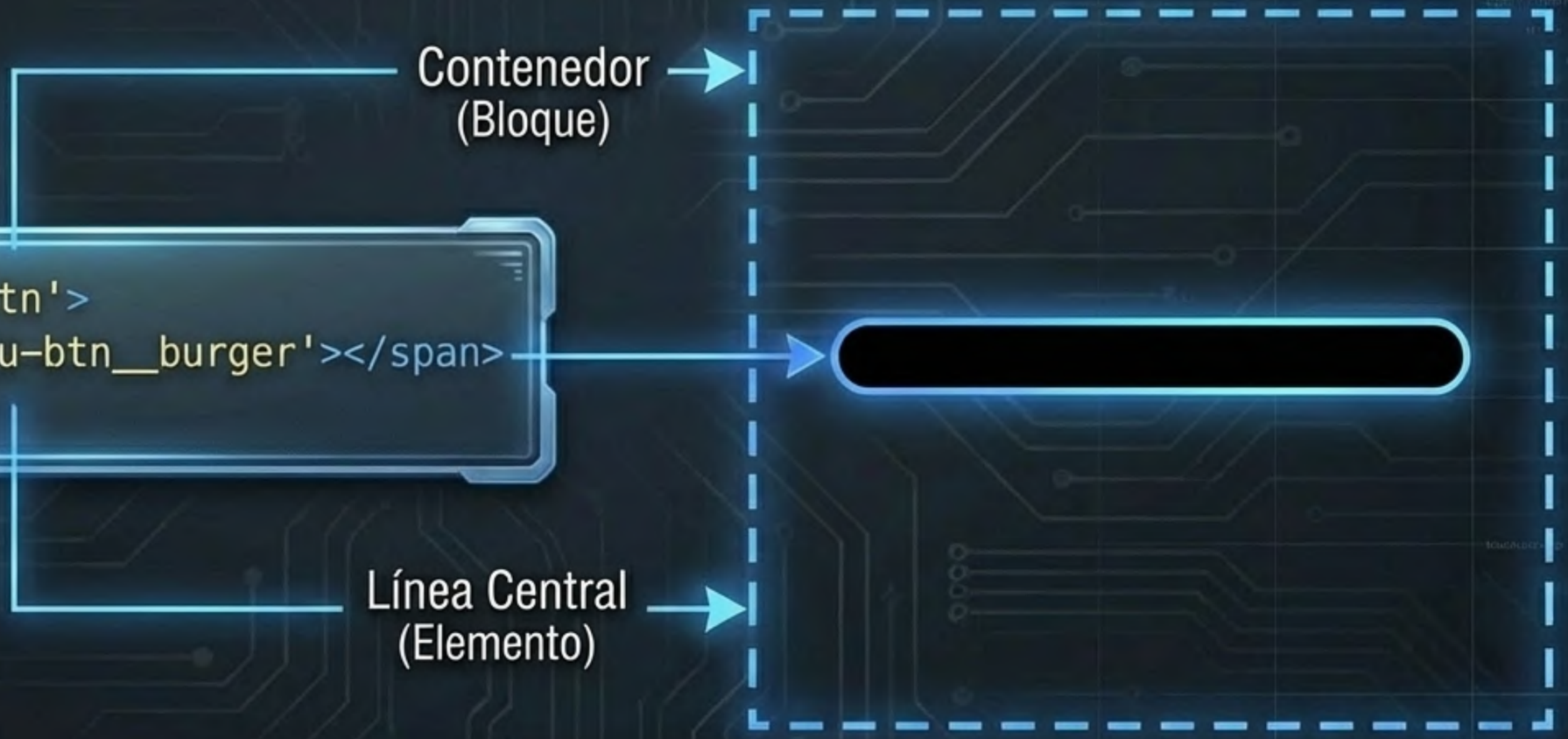
- ☑ Variables (Colores y Tamaños)
- ☑ Anidamiento & Ampersand (Estructura)
- ☑ Mixins (Transiciones suaves)
- ☑ Modificadores BEM (Estado .is-active)

NotebookLM

Paso 1: La Estructura BEM (HTML)

El menú no requiere imágenes. Usaremos un contenedor principal y una sola línea central.
Las líneas superior e inferior serán pseudo-elementos creados en SCSS.

```html
<div class='menu-btn'>
  <span class='menu-btn__burger'></span>
</div>
```



NotebookLM

Paso 2: SCSS Base y Variables

Definimos el contenedor y usamos el operador & para estilizar la línea central (__burger) sin repetir el nombre del bloque.

```scss
$color-menu: #333;

.menu-btn {
    position: relative;
    width: 28px;
    height: 20px;

    &__burger {
        position: absolute;
        width: 28px; height: 3px;
        background: $color-menu;
    }
}
```

NotebookLM

Paso 3: Los Panes (Pseudo-elementos)

Anidamos &::before y &::after dentro del elemento __burger. Usamos translate en el eje Y para posicionarlos arriba y abajo.

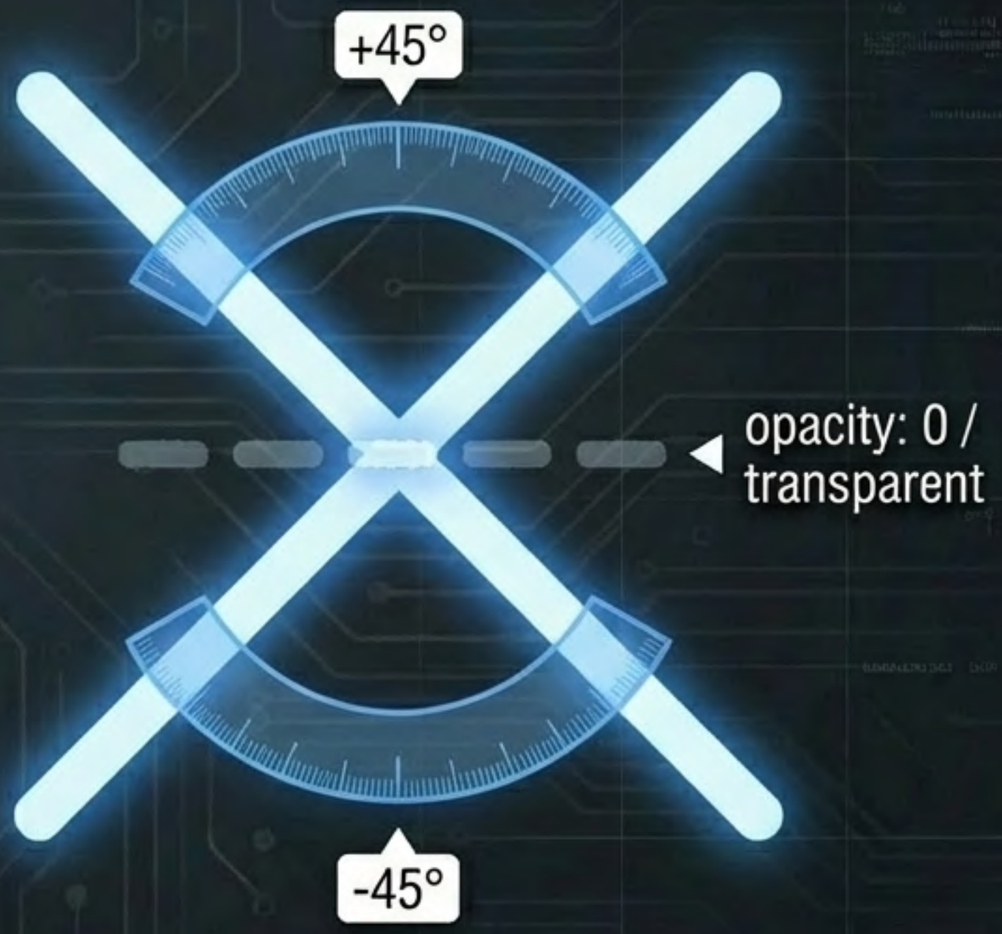
```
&::before, &::after {
    content: '';
    position: absolute;
    width: 28px; height: 3px;
  }

  &::before { transform: translateY(-8px); }
  &::after  { transform: translateY(8px); }
  &::after  { transform: translateY(8px); }
```

NotebookLM

Paso 4: La Animación (El Modificador BEM)

Al agregar la clase --activo, el SCSS oculta la línea central y rota los pseudo-elementos 45 grados para formar una 'X'.

```
&--activo {
  .menu-btn__burger {
    background: transparent;
    &::before { transform: rotate(45deg); }
    &::after { transform: rotate(-45deg); }
  }
}
```



NotebookLM

Síntesis: El Flujo de Trabajo SCSS + BEM

Escribir CSS modular ya no tiene que ser doloroso. Con SCSS, tu código es limpio, mantenible y lógicamente estructurado.

Tokens (Variables)

```
$color-marca: #00FFCC;
```

Almacena valores globales

Estructura (Anidamiento + &)

```
.bloque {
  &__elemento { } }
}
```

Mapea directamente al HTML BEM

Lógica (Mixins)

```
@include transicion-suave;
```

Automatiza reglas repetitivas

Módulos (@use)

_boton.scss main.scss

Organiza en archivos parciales

NotebookLM